

› LAB 1| 22/01/26

↳ 8 cells hidden

› LAB 2| 29/01/26

↳ 7 cells hidden

› LAB 3| 05/02/26

↳ 7 cells hidden

› LAB 4| 12/02/2026

↳ 12 cells hidden

› LAB 5

↳ 1 cell hidden

› LAB 7

↳ 2 cells hidden

› LAB 8| 12 March 2026

↳ 6 cells hidden

› Add next layer using reloc

↳ 6 cells hidden

› LAB 9 (19/03/2026)

↳ 4 cells hidden

✓ LAB 10 (02/04/26)

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Input

import kagglehub
import os

# Download dataset
path = kagglehub.dataset_download("vikrishnan/iris-dataset")
print("Path to dataset files:", path)

# 🐞 Check actual file name (VERY IMPORTANT)
files = os.listdir(path)
print("Files in dataset:", files)

# Load dataset (automatically picks correct file)
df = pd.read_csv(os.path.join(path, files[0]))
```

```
# Preview data
print(df.head())

# Features and labels
X = df.iloc[:, :-1].values
y = df.iloc[:, -1].values

# Encode labels (setosa, versicolor, virginica → 0,1,2)
le = LabelEncoder()
y = le.fit_transform(y)

# Normalize features
scaler = StandardScaler()
X = scaler.fit_transform(X)

# Train-test split (fixed)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Build model
model = Sequential([
    Input(shape=(4,)),
    Dense(16, activation='relu'),
    Dense(16, activation='relu'),
    Dense(3, activation='softmax')
])

# Compile model
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

# Summary
model.summary()

# Train model
model.fit(
    X_train, y_train,
    epochs=50,
    batch_size=8,
    validation_split=0.1,
    verbose=1
)

# Evaluate model
test_loss, test_acc = model.evaluate(X_test, y_test)

print("Test accuracy:", test_acc)
```


Using Colab cache for faster access to the 'iris-dataset' dataset.
 Path to dataset files: /kaggle/input/iris-dataset
 Files in dataset: ['iris.data.csv']
 5.1 3.5 1.4 0.2 Iris-setosa
 0 4.9 3.0 1.4 0.2 Iris-setosa
 1 4.7 3.2 1.3 0.2 Iris-setosa
 2 4.6 3.1 1.5 0.2 Iris-setosa
 3 5.0 3.6 1.4 0.2 Iris-setosa
 4 5.4 3.9 1.7 0.4 Iris-setosa
 Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 16)	80
dense_1 (Dense)	(None, 16)	272
dense_2 (Dense)	(None, 3)	51

Total params: 403 (1.57 KB)
 Trainable params: 403 (1.57 KB)
 Non-trainable params: 0 (0.00 B)

Epoch 1/50
 14/14 ————— 2s 30ms/step - accuracy: 0.5514 - loss: 0.9880 - val_accuracy: 0.5833 - val_loss: 0.9406
 Epoch 2/50
 14/14 ————— 0s 12ms/step - accuracy: 0.6355 - loss: 0.9113 - val_accuracy: 0.5833 - val_loss: 0.8525
 Epoch 3/50
 14/14 ————— 0s 6ms/step - accuracy: 0.6636 - loss: 0.8426 - val_accuracy: 0.6667 - val_loss: 0.7746
 Epoch 4/50
 14/14 ————— 0s 7ms/step - accuracy: 0.6636 - loss: 0.7831 - val_accuracy: 0.6667 - val_loss: 0.7081
 Epoch 5/50
 14/14 ————— 0s 7ms/step - accuracy: 0.6822 - loss: 0.7285 - val_accuracy: 0.6667 - val_loss: 0.6539
 Epoch 6/50
 14/14 ————— 0s 6ms/step - accuracy: 0.7103 - loss: 0.6792 - val_accuracy: 0.7500 - val_loss: 0.6113

```
# 1. Import Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import os
import kagglehub

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Input

# 2. Load Dataset
path = kagglehub.dataset_download("vikrishnan/iris-dataset")
print("Path:", path)

files = os.listdir(path)
print("Files:", files)

df = pd.read_csv(os.path.join(path, files[0]))
print(df.head())

# 3. Features & Labels
X = df.iloc[:, :-1].values
y = df.iloc[:, -1].values

# 4. Encode Labels
le = LabelEncoder()
y = le.fit_transform(y)

# 5. Normalize
scaler = StandardScaler()
X = scaler.fit_transform(X)

# 6. Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# 7. Build Model (ANN)
model = Sequential([
    Input(shape=(4,)),
    Dense(16, activation='relu'),
    Dense(16, activation='relu'),
    Dense(3, activation='softmax')
])

# 8. Compile
model.compile()
```

```
optimizer='adam',
loss='sparse_categorical_crossentropy',
metrics=['accuracy']
)

# 9. Summary
model.summary()

# 10. Training (with history like MNIST)
history = model.fit(
    X_train, y_train,
    epochs=50,
    batch_size=8,
    validation_split=0.1
)

# 11. Evaluation
test_loss, test_acc = model.evaluate(X_test, y_test)
print("Test Accuracy:", test_acc)

# 12. Graphs (Same as MNIST)

# Accuracy Graph
plt.figure()
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('Model Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend(['Train', 'Validation'])
plt.show()

# Loss Graph
plt.figure()
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title('Model Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend(['Train', 'Validation'])
plt.show()
```